

Title

Design and Develop a Real-Time Service Request Management System

Business Context

A growing service organization currently manages customer service requests manually, resulting in delayed updates and poor visibility into request progress. The organization requires a web-based system that enables operators to submit service requests while supervisors monitor processing in real time.

The application should support multiple requests being processed simultaneously, provide live status updates to connected users, and remain responsive while long-running operations execute in the background.

The objective of this assignment is to evaluate the candidate's ability to analyze business requirements, design an appropriate solution, and implement a working full-stack application using technologies of their choice.

Candidate Objectives

- The candidate is expected to:
- Analyze the business problem.
- Define functional and non-functional requirements.
- Design the overall solution.
- Justify the selected technology stack.
- Implement the application.
- Demonstrate software engineering best practices.
- Present assumptions and design decisions.

Mandatory Technical Skills to Demonstrate

The implementation **must** demonstrate the following technical capabilities:

Backend

- RESTful API development
- Database design and integration
- Input validation
- Error handling
- Layered application architecture
- Configuration management

Frontend

- Responsive user interface
- API integration
- Form validation
- State management
- Real-time UI updates

Real-Time Communication (Mandatory)

The system **must** use **WebSockets** (or an equivalent real-time communication protocol) to push updates from the server to connected clients.

Polling is not acceptable as the primary update mechanism.

Concurrent Processing (Mandatory)

The system **must** demonstrate concurrent execution using **threads, background workers, asynchronous tasks, or an equivalent concurrency mechanism** supported by the chosen technology stack.

Long-running processes must not block incoming API requests.

Database

- Appropriate schema design
- Data persistence
- Basic relationships and constraints

Software Engineering Practices

- Git version control
- Meaningful commit history
- Clear project structure
- Documentation

Required Deliverables

1. System Analysis

Prepare a short analysis document that includes:

Problem Statement

- Business objectives
- Users
- Assumptions
- Scope

Functional Requirements

Examples include:

- Create service request
- View requests
- Update request status
- Monitor request progress
- Receive live updates
- Cancel requests
- Search and filter requests

The candidate should identify and define the complete set of functional requirements.

Non-Functional Requirements

Examples include:

- Performance
- Scalability
- Reliability
- Security
- Maintainability
- Availability
- Usability
- Responsiveness

The candidate should identify and justify the applicable non-functional requirements.

2. System Design

Include:

- High-level architecture diagram
- Component diagram
- Database design (ERD or schema)
- API design
- WebSocket communication flow
- Concurrency model
- Technology stack justification

3. Implementation

Implement the planned solution using any preferred technology stack.

The implementation should align with the submitted analysis and design.

4. Documentation

Provide a README containing:

- Setup instructions
- Architecture overview
- Technology stack
- Assumptions
- Build and run instructions
- API documentation
- WebSocket events
- Database setup

Technical Constraints

The candidate may choose any suitable technology stack.

The completed solution **must** include:

- REST APIs
- WebSocket communication
- Concurrent/background processing
- Persistent data storage
- Responsive web interface

Evaluation Framework

Evaluation Area	Assessment Criteria	Weight
Business & System Analysis	Understanding of the problem, assumptions, scope definition, completeness of functional and non-functional requirements	15%
Solution Design	Architecture, modularity, database design, API design, concurrency approach, WebSocket design, justification of technical decisions	15%
Backend Implementation	API quality, validation, error handling, business logic, database integration, code organization	20%
Frontend Implementation	User experience, responsiveness, component design, API integration, state management	15%
Real-Time Communication	Correct implementation of WebSockets, efficient event handling, automatic client updates without polling	10%
Concurrency & Background Processing	Correct use of threads, asynchronous tasks, or workers; handling of multiple simultaneous operations; thread safety where applicable	10%
Code Quality & Engineering Practices	Readability, maintainability, project structure, Git usage, naming conventions, separation of concerns	10%
Documentation & Presentation	Quality of README, setup instructions, API documentation, architecture documentation, clarity of explanations	5%

Total: 100%

Bonus Considerations (Optional)

Exceptional submissions may receive additional recognition (without altering the 100% evaluation score) for features such as:

- Automated testing
- Authentication and authorization
- Docker or containerization
- CI/CD pipeline
- Monitoring or logging
- Deployment to a cloud platform
- Caching
- Advanced search or filtering
- Role-based access control
- Comprehensive test coverage

Expected Outcome

At the conclusion of the assignment, the candidate should have demonstrated the ability to:

- Translate a business problem into software requirements.
- Produce a structured software design before implementation.
- Build a complete full-stack application using a technology stack of their choice.
- Implement real-time communication using WebSockets.
- Implement concurrent processing using threads or equivalent background execution mechanisms.
- Deliver a maintainable, documented, and production-minded solution that reflects sound software engineering principles.